

MixBytes()

[SECURITY AUDIT REPORT]

Lido Oracle v8.2.0 Diff

PREPARED¹ FOR LIDO

REPORT DATE

October 07, 2026

Table of Contents

1. Introduction	3
1.1 Disclaimer	3
1.2 Executive Summary	3
1.3 Project Overview	5
1.4 Security Audit Methodology	7
1.5 Risk Classification	9
1.6 Summary of Findings	10
2. Findings Report	10
2.1 Critical	10
2.2 High	10
2.3 Medium	10
2.4 Low	10
3. About MixBytes	11

1.1 Disclaimer

The audit makes no statements or warranties regarding the utility, safety, or security of the code, the suitability of the business model, investment advice, endorsement of the platform or its products, the regulatory regime for the business model, or any other claims about the fitness of the contracts for a particular purpose or their bug-free status.

1.2 Executive Summary

The Lido Oracle is an off-chain daemon for the Lido protocol that observes state across the Execution and Consensus layers and submits periodic reports to Lido's on-chain contracts through a hash-consensus flow among a quorum of oracle members. This engagement covered the changes between release `8.1.0` (commit `032c228c767759e67da43e6c40fa81732257879d`, the last audited version, reviewed by Composable Security) and release `8.2.0` (commit `fff59c898eebbcc41790b05b03e884d2f5f53fc0`, the audit target), plus verification of the published `8.2.0` Docker image. The change set introduces a new oracle instance for the Community Staking Module `0x02` - an eight-line subclass of the existing staking-module performance oracle with `COMPATIBLE_CONTRACT_VERSION = 3` and `COMPATIBLE_CONSENSUS_VERSION = 4` - together with its dispatch entrypoint, a refactor of the environment-variable validation from per-module checks to a `STAKING_MODULE_ORACLES` set, a `docker-compose` service definition, and an observability fix that logs the full traceback for Web3 exceptions swallowed by the cycle handler.

The interim security review was conducted over 1 working day via manual code review and a proprietary AI-assisted analysis tool.

Because the daemon's outputs are consumed by on-chain contracts, the review spanned both the changed Python code and its interaction points with the Lido contracts the oracle reports into. Alongside the general checklist for off-chain services (input validation, exception handling, configuration validation, secret handling, and failure modes under adversarial input), the following areas were investigated in depth:

- **Dispatch and configuration completeness.** Every code path that special-cases the existing `csm` and `cm` staking-module oracles was enumerated to establish that the new `csm_0x02` instance is handled everywhere it must be: module dispatch in `main.py`, environment-variable validation in `variables.py`, and the compose wiring. No remaining per-module check bypasses the new instance.
- **Compatibility constants against the deployed contracts.** The subclass pins `COMPATIBLE_CONTRACT_VERSION = 3` and `COMPATIBLE_CONSENSUS_VERSION = 4`. Both values were cross-checked against the actually deployed CSM `0x02` contract set verified in the companion deployment-verification engagement (contracts initialized at version 3, FeeOracle consensus version 4), and against the sibling `csm` and `cm` oracles, which pin the same pair. The version gate in the consensus module rejects reports with `<=` semantics and requires exact equality before reporting, so a mismatch would surface as a loud wait state rather than a misreport.
- **The delegation-contract wiring.** The new compose service passes `DELEGATION_CONTRACT_ADDRESS` to the signer module (the EIP-7251 delegation used for `0x02`-credential validator withdrawals). The empty-string default was traced through `SignerModule` to confirm delegation is cleanly disabled when unset, matching the sibling services.

- **The traceback-logging change.** The full exception chain is now serialized into the structured log for swallowed `Web3Exception`s. The format call was checked on the project's Python 3.14, the `traceback` import confirmed, and the log content reviewed for exposure of secrets (provider URLs were already logged at startup; no new exposure surface is introduced).

Out of scope of this audit is the pre-existing oracle machinery that the new instance reuses without modification - the `SMPerformanceOracle` base class, the consensus, signer, transaction, and provider modules, and the Merkle-tree flow through the performance collector - all of which were reviewed by Composable Security in the [8.1.0](#) report and by MixBytes at earlier versions; this engagement reviewed only how the new instance plugs into them. The on-chain CSM 0x02 contracts themselves (`CSModule` , `Accounting` , `FeeOracle` , `HashConsensus` , and the rest of the module set at commit `4d3de665`) are covered by the separate deployment-verification engagement and third-party audits, and were used here only as the reference for the compatibility constants. The Keys API service and the fork-test infrastructure changes are likewise outside the scope.

Below we set out our overall assessment, key assumptions, and main recommendations.

- **The change set is minimal, correct, and configuration-level.** The new oracle instance adds no logic: it reuses the audited staking-module performance machinery and differs from the existing CSM oracle only in its name and endpoint wiring. No vulnerability was identified in the reviewed diff; no findings are reported.
- **The version constants match the deployed CSM 0x02 contracts exactly.** Contract version 3 and consensus version 4 were independently verified on-chain during the companion deployment verification, and the consensus gate enforces exact equality before any report is submitted, so the oracle cannot silently report against an upgraded contract.
- **The observability fix improves on a previously reported gap.** Full tracebacks for exceptions swallowed by the cycle handler partially address the silent-failure visibility concern raised in the [8.0.6](#) report; the underlying silent-drop behaviour in the gas-clamp path, which the client acknowledged at the time, is unchanged and remains out of this diff's scope.

1.3 Project Overview

Summary

CLIENT	Lido
CATEGORY	Liquid Staking
PROJECT	Lido Oracle
TYPE	Python
PLATFORM	EVM
TIMELINE	06.10.2026 - 07.10.2026

Scope of Audit

src/main.py

src/types.py

src/variables.py

src/modules/oracles/common/oracle_module.py

src/modules/oracles/staking_modules/community_staking_0x02/__init__.py

src/modules/oracles/staking_modules/community_staking_0x02/csm_0x02.py

src/modules/oracles/staking_modules/community_staking_0x02/entrypoint.py

The scope of this engagement is the diff between commit `032c228c767759e67da43e6c40fa81732257879d` (release `8.1.0`, the last audited version) and commit `fff59c898eebbcc41790b05b03e884d2f5f53fc0` (release `8.2.0`, the audit target).

Versions Log

DATE	COMMIT HASH	NOTE
07.10.2026	032c228c767759e67da43e6c40fa81732257879d	Diff Base
07.10.2026	fff59c898eebbcc41790b05b03e884d2f5f53fc0	Initial Commit

Docker Image Hash Validation

The release is published as `lidofinance/oracle:8.2.0`, manifest digest `sha256:ccebb33e651983a90d3bb2ccce733f27847a38ba51211a3d87f28a5738f76227`, image (config) digest `sha256:3e268ac920ff5b12a058d2c55793e05afc21bd4bb8708cc17e8d59736ed9442c`. It was built from tag `8.2.0`, which points at commit `fff59c898eebbcc41790b05b03e884d2f5f53fc0`, the commit reviewed in this report.

The image was pulled by digest and verified against a local `linux/amd64` build of the audit commit as follows. The application tree (`/app`) inside the published image is file-for-file identical to the repository at `fff59c898eebbcc41790b05b03e884d2f5f53fc0` , the only difference being the provenance file `build-info.json` that the release pipeline adds. All pinned virtualenv dependencies match the lockfile-installed set of the local build.

1.4 Security Audit Methodology

Stage 1 Interim Audit

01 PROJECT ARCHITECTURE REVIEW

OBJECTIVE: UNDERSTAND THE OVERALL STRUCTURE OF THE PROTOCOL AND IDENTIFY POTENTIAL SECURITY RISKS, INCLUDING THE PRIMITIVES AND ABSTRACTIONS IMPLEMENTED BY THE SYSTEM AND WHERE DESIGN OR INTEGRATION WEAKNESSES COULD BE EXPLOITED.

- Understanding overall system design and contract interactions
- Mapping responsibilities of each component and protocol workflows
- Conducting a thorough line-by-line review of contracts and modules
- Tracing execution paths and mapping state transitions
- Identifying trust boundaries and external integrations
- Forming an independent architectural understanding directly from code before validating documentation
- Running the project test suite to observe behavior and encoded invariants

02 ADVERSARIAL CODE REVIEW

OBJECTIVE: IDENTIFY POTENTIAL VULNERABILITIES SPECIFIC TO THE PROTOCOL ARCHITECTURE, BUSINESS LOGIC, AND ECONOMIC MECHANISMS.

- Analyzing the codebase from an attacker's perspective: what can fail, not only what works
- Searching for edge cases, invalid assumptions, unexpected call sequences, and unsafe state transitions
- Attempting to violate protocol invariants and find where guarantees break
- Analyzing economic attack surfaces: oracle dependencies, share pricing, access control, cross-contract interactions
- Writing targeted tests, modelling adversarial scenarios, mutation testing, fuzzing, and PoC exploits
- Independent exploit-path discovery by each auditor to reduce anchoring bias
- Collaborative pair-auditing of high-risk code led by the Team Lead

03 SYSTEMATIC VULNERABILITY ANALYSIS

OBJECTIVE: APPLY STRUCTURED ANALYSIS AND KNOWN ATTACK PATTERNS TO ENSURE COMPREHENSIVE COVERAGE ACROSS THE CODEBASE.

- Reviewing code against an internally maintained vulnerability checklist derived from past exploits and research
- Analyzing common DeFi attack classes: reentrancy, accounting manipulation, oracle manipulation, privilege escalation, state desynchronization
- Using proprietary AI tooling to surface candidate issues across broader attack vectors
- Manually validating and triaging AI-generated candidates

04 CONSOLIDATION OF AUDITORS' REPORTS

OBJECTIVE: MERGE INTERIM INPUTS FROM ALL AUDITORS INTO A COHERENT REPORT WITH CONSISTENT SEVERITY LEVELS AND REPRODUCIBLE FINDINGS.

- Cross-checking findings across auditors; consolidating and deduplicating issues
- Resolving discrepancies in severity assessments
- Using proprietary AI tooling to assist with report drafting and consistency checks
- Manually reviewing and finalizing the report narrative for client review

Stage 2

Re-audit & Mainnet Verification

RE-AUDIT

OBJECTIVE: VERIFY THAT CLIENT REMEDIATIONS ARE CORRECTLY IMPLEMENTED AND THE UPDATED CODE INTRODUCES NO NEW VULNERABILITIES.

- Confirming each remediation matches the original recommendation
- Documenting rationale for any unresolved findings
- Verifying modified logic and integration points remain secure
- Executing the test suite after remediation, targeting fixed areas
- Running AI tooling on the updated codebase

MAINNET DEPLOYMENT VERIFICATION

OBJECTIVE: FINAL VERIFICATION OF DEPLOYED CONTRACTS AND CONFIGURATION BEFORE ISSUING THE PUBLIC REPORT.

- Verifying deployed bytecode against audited source across networks
- Matching bytecode to the build from the audited commit, identical compiler settings
- Reviewing constructor and initializer arguments
- Verifying proxy order, initialization flow, and admin configuration
- Ensuring implementations aren't left uninitialized or exposed to init front-running

1.5 Risk Classification

Severity Level Matrix

SEVERITY	IMPACT: HIGH	IMPACT: MEDIUM	IMPACT: LOW
LIKELIHOOD: HIGH	CRITICAL	HIGH	MEDIUM
LIKELIHOOD: MEDIUM	HIGH	MEDIUM	LOW
LIKELIHOOD: LOW	MEDIUM	LOW	LOW

IMPACT

HIGH - Theft exceeding 0.5% of the protocol's TVL, partial or complete blocking of funds on the contract without the possibility of withdrawal (>0.5%), or loss of user funds exceeding 1% for users interacting with the protocol.

MEDIUM - Contract lock that can only be resolved through a contract upgrade, one-time theft of rewards or an amount up to 0.5% of the protocol's TVL, or funds lock where withdrawal is still possible by an administrator.

LOW - One-time contract lock that can be resolved by an administrator without requiring a contract upgrade.

LIKELIHOOD

HIGH - An event with an estimated 50–60% probability of occurring within a year, which can be triggered by any actor (e.g. due to a market condition that the actor cannot influence).

MEDIUM - An unlikely event (10–20% probability of occurring) that can be triggered by a trusted actor.

LOW - A highly unlikely event that can only be triggered by the contract owner.

ACTION REQUIRED

CRITICAL - Must be fixed as soon as possible.

HIGH - Strongly advised to be fixed in order to minimize potential risks.

MEDIUM - Recommended to be fixed to improve security and stability.

LOW - Recommended to be fixed to improve overall robustness and efficiency.

FINDING STATUS

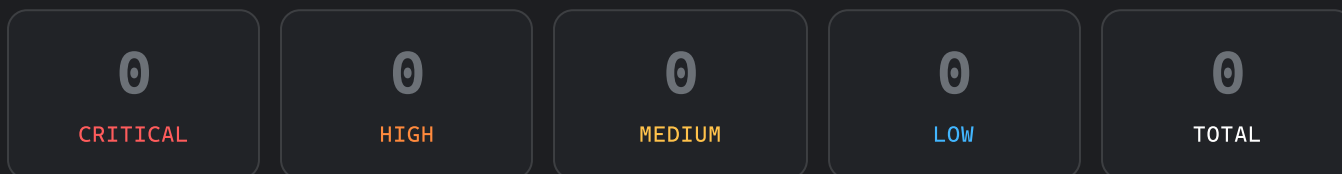
● **FIXED** - The recommended fixes have been implemented and the issue no longer impacts the security of the protocol.

● **PARTIALLY FIXED** - The fixes have been partially implemented, reducing the impact, but the issue has not been fully resolved.

● **ACKNOWLEDGED** - The fixes were not implemented; the finding is unresolved or was accepted by the team without code changes.

1.6 Summary of Findings

Findings Count



Findings Statuses

All finding statuses reflect the state of the code at the latest revision listed in the Versions Log.

2.1 Critical

NO CRITICAL SEVERITY FINDINGS

2.2 High

NO HIGH SEVERITY FINDINGS

2.3 Medium

NO MEDIUM SEVERITY FINDINGS

2.4 Low

NO LOW SEVERITY FINDINGS

Built on Trust

MixBytes is a blockchain security firm specializing in the analysis of decentralized protocols and smart contract systems. The company helps Web3 teams build resilient protocol architecture, smart contract logic, and economic mechanisms through a combination of AI-assisted analysis and senior human expertise.

Rather than focusing solely on one-time audits before deployment, MixBytes works with protocols across their entire lifecycle - from early architecture design to production audits and ongoing protocol evolution. Our team consists of experienced security researchers, engineers, and protocol analysts with deep expertise in DeFi systems, adversarial protocol analysis, and smart contract security.

Over the years, MixBytes has worked with many of the most widely used protocols in the ecosystem, including Lido, Aave, Curve, 1inch, OKX, Mantle, Fluid, Gearbox, Resolv, and others. To enhance analysis efficiency and coverage, MixBytes also develops and uses internal AI-assisted tooling that helps surface potential risk signals during development and code review.

SECURED TVL

\$50B+

AUDITS DELIVERED

300+

WEB3 PROTOCOLS

80+

Protocol Security Lifecycle

MixBytes supports protocols across the full lifecycle of their development and operation.

Web-3 Design Review

Independent assessment of protocol architecture and economic design before critical decisions are embedded in production code.

AI Tooling

AI-assisted analysis integrated into development workflows to surface potential risk signals during protocol development.

Smart Contract Audit

Comprehensive manual verification of smart contract logic and protocol invariants before production deployment.

Security Support

Continuous expert support for protocol upgrades, integrations, governance changes, and evolving attack surfaces after launch.

CONTACTS

hello@mixbytes.io

[X](#) [Telegram](#) [Discord](#) [LinkedIn](#)